SYSTEM DESIGN & SOFTWARE ARCHITECTURE

AI Academic Mentor

Team Name	CSE4104-7C-T05
Section	7C
Project Title	AI Academic Mentor — Personalized Learning & Study Assistant
Course Code	CSE 4104
Document Type	Week 4 — System Design
Date	15 June 2026
Instructed By	Md. Riaz Mahmud, Assistant Professor, CSE Dept., NUBTK
GitHub	https://github.com/Priyo-1158/cse4104-7c-t05-ai-academic-mentor

TEAM INFORMATION

NAME	STUDENT ID	ROLE
Shadman Sadequeen Priyo	11230121158	Team Leader
Musabbir Hossain Chayon	11230121168	Frontend Developer
Sk. Nadirul Haque Nadir	11230121155	Backend Developer
Shafin Kabir	11230121175	AI Integration Lead

Project Information

The AI Academic Mentor is a personalized learning and study assistant designed to support university students with intelligent, on-demand academic guidance. Developed by Team CSE4104-7C-T05 (Section 7C, NUBTK) under the supervision of *Md. Riaz Mahmud, Assistant Professor, CSE Dept.*, the project integrates modern web technologies with advanced AI capabilities to enhance student productivity and learning outcomes.

The platform follows a three-tier architecture — presentation, backend API, and database — with an additional AI inference layer powered by Google Gemini 2.0 Flash. This ensures secure, real-time academic assistance while maintaining strict authentication and rate-limiting protocols. Students can interact with the system through features such as AI-powered chat sessions, adaptive quiz generation, note summarization, and personalized study planning, all of which are stored and managed via MongoDB Atlas for scalability and reliability.

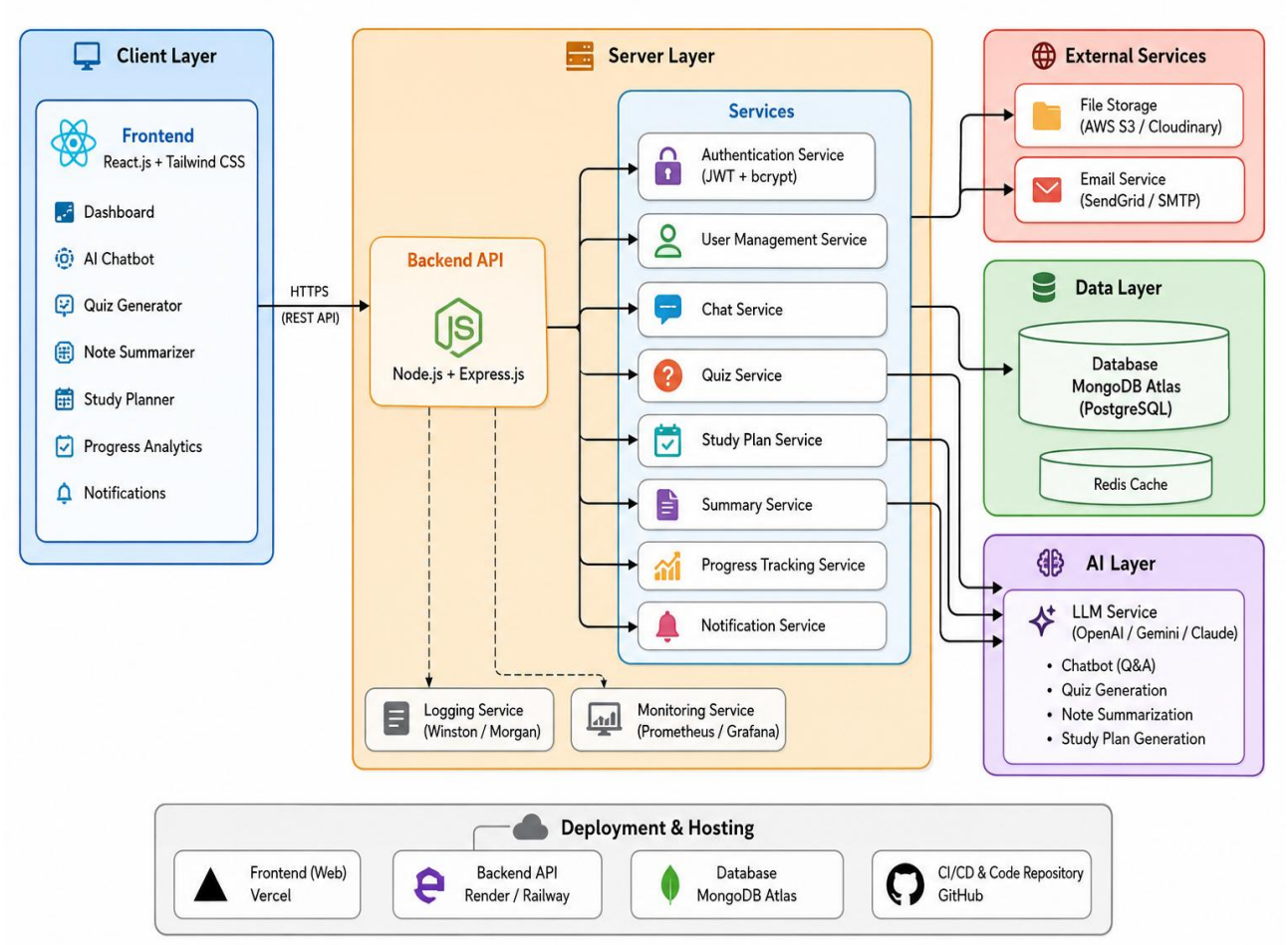
The project's primary goal is to address three critical academic challenges: lack of instant support outside lecture hours, limited self-assessment opportunities, and inefficiency in consolidating dense lecture notes. By leveraging AI, the system provides 24/7 contextual academic responses, unlimited quizzes, structured note summaries, and tailored study schedules. This combination of Node.js, Express.js, MongoDB, and Gemini AI positions the platform as a robust prototype for future academic mentoring systems, aligning with both course objectives and real-world educational needs.

TABLE OF CONTENTS

1. System Architecture

1.1 Overview

The AI Academic Mentor platform follows a three-tier web application architecture, decoupling presentation, business logic, and data persistence. A fourth layer — the AI inference tier — communicates with Google Gemini 2.0 Flash via a secure backend proxy, ensuring API keys are never exposed to the client.



1.2 Architecture Diagram

The following diagram illustrates the full system topology, data flows, and inter-component communication pathways:

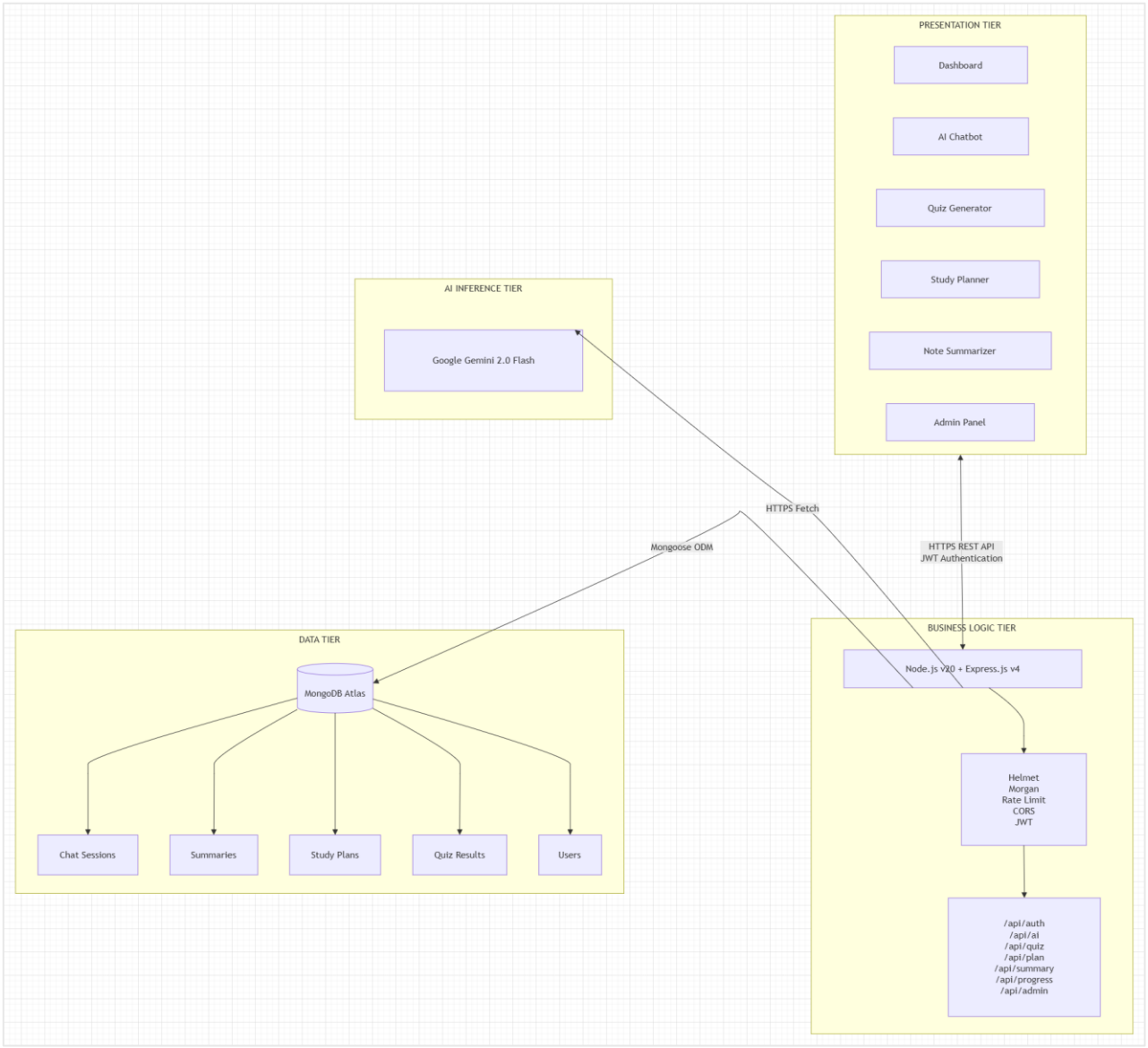


Figure: Architecture Diagram

1.3 Technology Stack

The following table enumerates each architectural component alongside its selected technology, justification, and deployment target:

Layer	Technology	Responsibility	Deployment
Presentation	HTML5 + CSS3 + Vanilla JS	User interface — all pages: Dashboard, Chatbot, Quiz, Planner, Summarizer, Progress, Admin	Vercel / Netlify
Backend API	Node.js v20 + Express.js v4	REST API, JWT authentication, request validation, rate limiting, Gemini proxy	Render / Railway
Database	MongoDB 7 + Mongoose ODM	User accounts, quiz results, study plans, note summaries, progress analytics	MongoDB Atlas (Cloud)
AI Inference	Google Gemini 2.0 Flash API	Natural language responses, quiz generation, note summarization, study plan creation	Google Cloud (Managed)
Authentication	JWT + bcryptjs	Stateless session tokens (7-day expiry), password hashing (12 rounds)	Backend (stateless)
Security	Helmet + express-rate-limit	HTTP security headers, DDoS mitigation — 100 req/15min global; 20 req/min for AI	Middleware layer
Version Control	Git + GitHub	Source code management, collaboration, CI readiness	github.com

2. Entity-Relationship Diagram

2.1 Database Design Rationale

The AI Academic Mentor data layer is hosted on MongoDB Atlas, a fully managed cloud document store. MongoDB's flexible schema accommodates the semi-structured nature of AI-generated content (quiz questions, summarised notes, study plans) without requiring rigid relational migrations. Six primary collections are defined below.

2.2 Collections & Attributes

Collection: users

Field	Type	Constraint	Key	Description
_id	ObjectId	AUTO	**PK**	MongoDB auto-generated primary key
name	String	REQUIRED	—	Full display name (2–100 chars)
email	String	REQUIRED, UNIQUE	—	Lowercase, used as login identifier
password	String	REQUIRED	—	bcrypt hash (12 rounds)
studentId	String	OPTIONAL	—	University student ID
department	String	DEFAULT: CSE	—	Academic department
role	Enum	DEFAULT: student	—	Values: student | admin
isActive	Boolean	DEFAULT: true	—	Soft-delete flag
lastLogin	Date	OPTIONAL	—	Timestamp of most recent login
createdAt	Date	AUTO	—	Mongoose timestamp
updatedAt	Date	AUTO	—	Mongoose timestamp

Collection: quiz_results

Field	Type	Constraint	Key	Description
_id	ObjectId	AUTO	PK	Primary key
userId	ObjectId	REQUIRED	**FK → users._id**	Reference to authenticated student

topic	String	REQUIRED	—	Subject or topic of quiz
score	Number	REQUIRED	—	Percentage score (0–100)
numQuestions	Number	REQUIRED	—	Total questions in session
correct	Number	REQUIRED	—	Number of correct answers
difficulty	Enum	DEFAULT: medium	—	easy | medium | hard
type	Enum	DEFAULT: mcq	—	mcq | truefalse | mixed
date	Date	AUTO	—	ISO 8601 timestamp

Collection: study_plans

Field	Type	Constraint	Key	Description
_id	ObjectId	AUTO	PK	Primary key
userId	ObjectId	REQUIRED	**FK → users._id**	Reference to owning student
subjects	Array<String>	REQUIRED	—	List of subjects for the plan
days	Number	DEFAULT: 5	—	Plan duration in days
hoursPerDay	Number	DEFAULT: 3	—	Target study hours per day
goal	String	DEFAULT: revision	—	Study objective or goal
plan	Array<Object>	REQUIRED	—	AI-generated day-by-day schedule
createdAt	Date	AUTO	—	Creation timestamp

Collection: summaries

Field	Type	Constraint	Key	Description
_id	ObjectId	AUTO	PK	Primary key
userId	ObjectId	REQUIRED	**FK → users._id**	Reference to owning student
subject	String	OPTIONAL	—	Academic subject label
notes	String	REQUIRED	—	Raw student notes (min 50 chars)
summary	String	REQUIRED	—	AI-generated summary text
style	Enum	DEFAULT: concise	—	concise | detailed | bullets | exam
createdAt	Date	AUTO	—	Creation timestamp

Collection: chat_sessions

Field	Type	Constraint	Key	Description
_id	ObjectId	AUTO	PK	Primary key
userId	ObjectId	REQUIRED	**FK → users._id**	Reference to student
messages	Array<Object>	REQUIRED	—	Ordered chat turns: {role, content, timestamp}
createdAt	Date	AUTO	—	Session start time
updatedAt	Date	AUTO	—	Last message timestamp

2.3 Relationships Summary

Parent Collection	Child Collection	Cardinality	Foreign Key
users	quiz_results	One-to-Many	quiz_results.userId → users._id
users	study_plans	One-to-Many	study_plans.userId → users._id
users	summaries	One-to-Many	summaries.userId → users._id
users	chat_sessions	One-to-Many	chat_sessions.userId → users._id

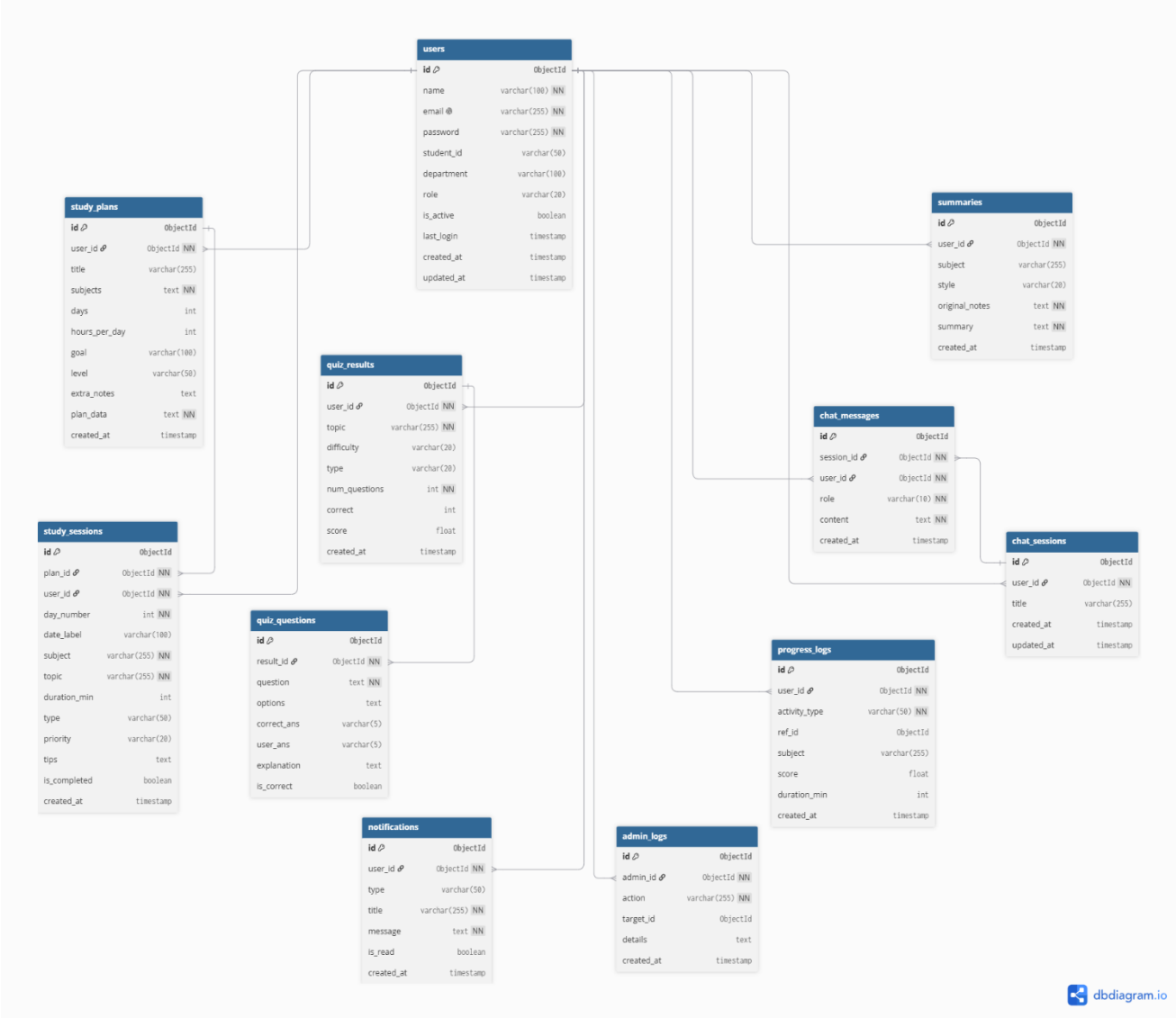


Figure: Entity-Relationship Diagram

3. Updated Use Case Diagram

3.1 Actors

The AI Academic Mentor system interacts with three distinct actor classes:

- Student (Primary Actor) — Registered university student who consumes all AI-powered academic features.
- Administrator (Secondary Actor) — Privileged user (role: admin) who manages the user base and monitors platform activity.
- Google Gemini AI (External System) — Third-party AI inference engine that processes natural language prompts and returns structured responses.

3.2 Use Case Matrix

Use Case	Student	Admin	Gemini AI
UC-01: Register Account	✓	—	—
UC-02: Login / Logout	✓	✓	—
UC-03: View Dashboard & Analytics	✓	—	—
UC-04: Conduct AI Chat Session	✓	—	✓
UC-05: Generate AI Quiz	✓	—	✓
UC-06: Submit Quiz & View Score	✓	—	—
UC-07: Summarize Notes via AI	✓	—	✓
UC-08: Generate Personalized Study Plan	✓	—	✓
UC-08a: View & Manage Study Planner	✓	—	—
UC-09: Track Study Progress	✓	—	—
UC-10: Manage Notification Preferences	✓	—	—
UC-11: Manage API Key Settings	✓	—	—
UC-12: View & Manage All Users	—	✓	—
UC-13: Monitor Platform Statistics	—	✓	—
UC-14: Deactivate / Manage Student Accounts	—	✓	—

3.3 Key Use Case Descriptions

UC-04: Conduct AI Chat Session

Precondition: Student is authenticated. Student submits a question or academic query via the chatbot interface. The backend validates the JWT, constructs a multi-turn conversation payload, and forwards it to the Gemini 2.0 Flash API with a system instruction that configures the model as a university-level academic assistant. The model returns a response which is displayed inline. The full conversation history is maintained client-side for contextual continuity.

UC-05: Generate AI Quiz

Precondition: Student is authenticated. Student specifies a topic, number of questions (default: 5), difficulty level (easy / medium / hard), and format (MCQ / True-False / mixed). The backend constructs a structured prompt instructing Gemini to return a valid JSON array of question objects. The response is parsed, validated, and rendered as an interactive quiz. On completion, the score and per-question explanation are displayed and results are persisted via POST /api/quiz.

UC-07: Summarise Notes via AI

Precondition: Student is authenticated and provides notes of at least 50 characters. Student selects a summarisation style: Concise (key points only), Detailed (comprehensive), Bullet Points (structured list), or Exam Mode (Key Concepts + Definitions + Quick Review). The Gemini model processes the notes with the chosen style directive and returns a structured summary, which is rendered and optionally saved.

UC-08: Generate Personalised Study Plan

Precondition: Student is authenticated. Student specifies one or more subjects, the plan duration in days (default: 5), daily study hours (default: 3), a goal descriptor (e.g. revision, exam prep, new material), and an optional difficulty level (beginner / intermediate / advanced). The backend constructs a structured prompt instructing Gemini to return a valid JSON array of day objects, each containing an ordered list of study sessions with subject, topic, duration in minutes, session type (lecture / practice / revision), priority level (high / medium / low), and study tips. The plan is rendered as a day-by-day interactive schedule and persisted via POST /api/plan. Students may view all previously generated plans from the planner history panel.

4. Activity Diagrams

4.1 Activity Diagram — AI Chat Workflow

The following activity diagram traces the complete lifecycle of an AI Chat interaction, from initial user input through authentication, rate-limit enforcement, AI inference, and final response rendering:

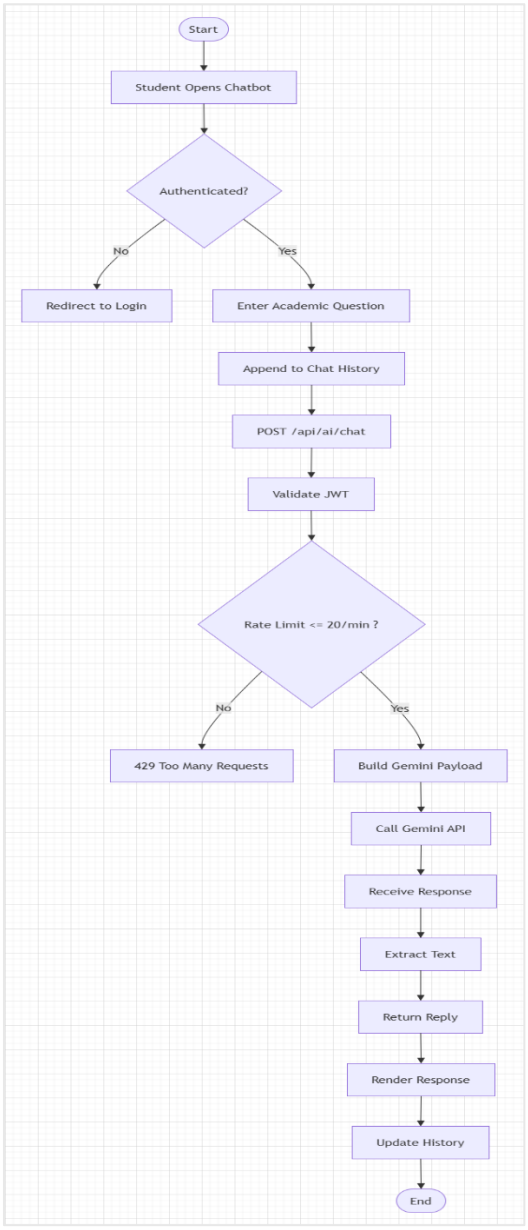


Figure: Activity Diagram — AI Chat Workflow

4.2 Activity Diagram — User Registration & Login

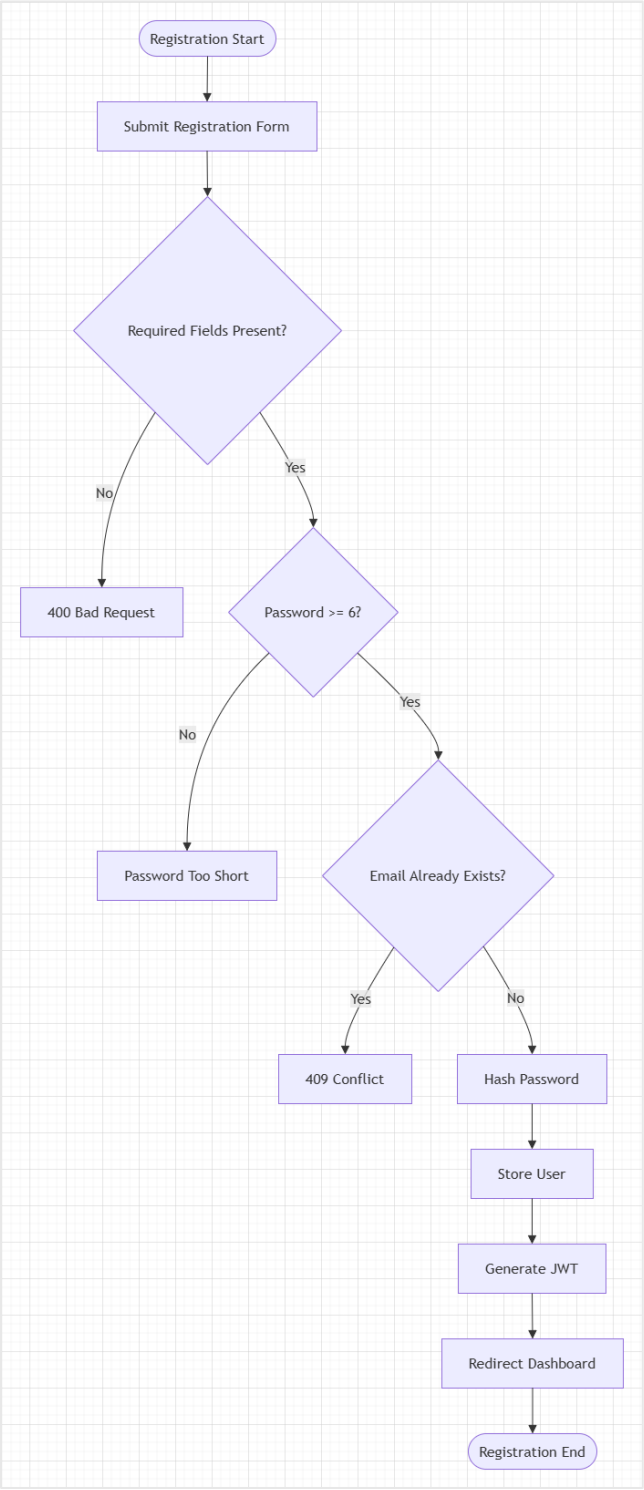


Figure: Activity Diagram — User Registration & Login

4.3 Activity Diagram — AI Quiz Generation Workflow

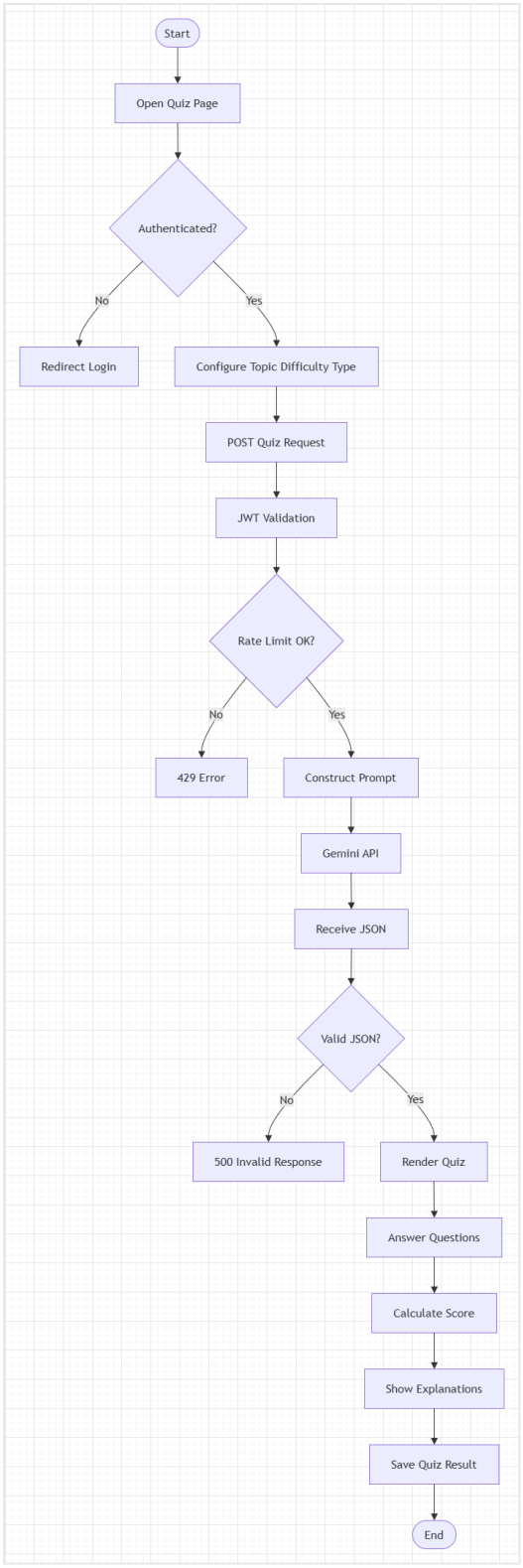


Figure: Activity Diagram — AI Quiz Generation Workflow

4.4 Activity Diagram — AI Note Summarizer Workflow

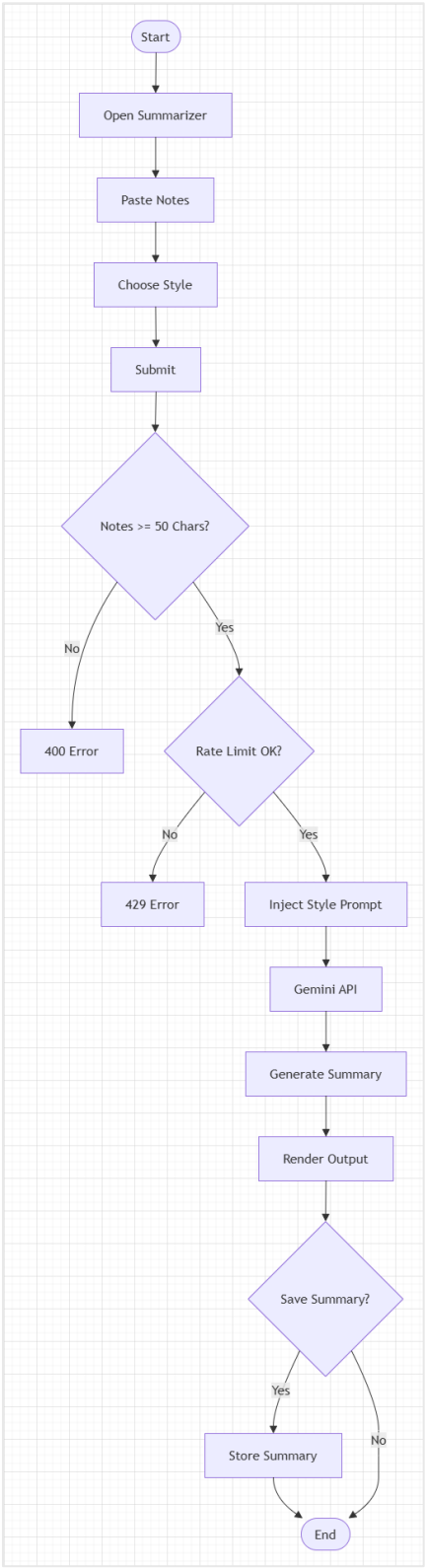


Figure: Activity Diagram — AI Note Summarizer Workflow

4.5 Activity Diagram — AI Study Planner Workflow

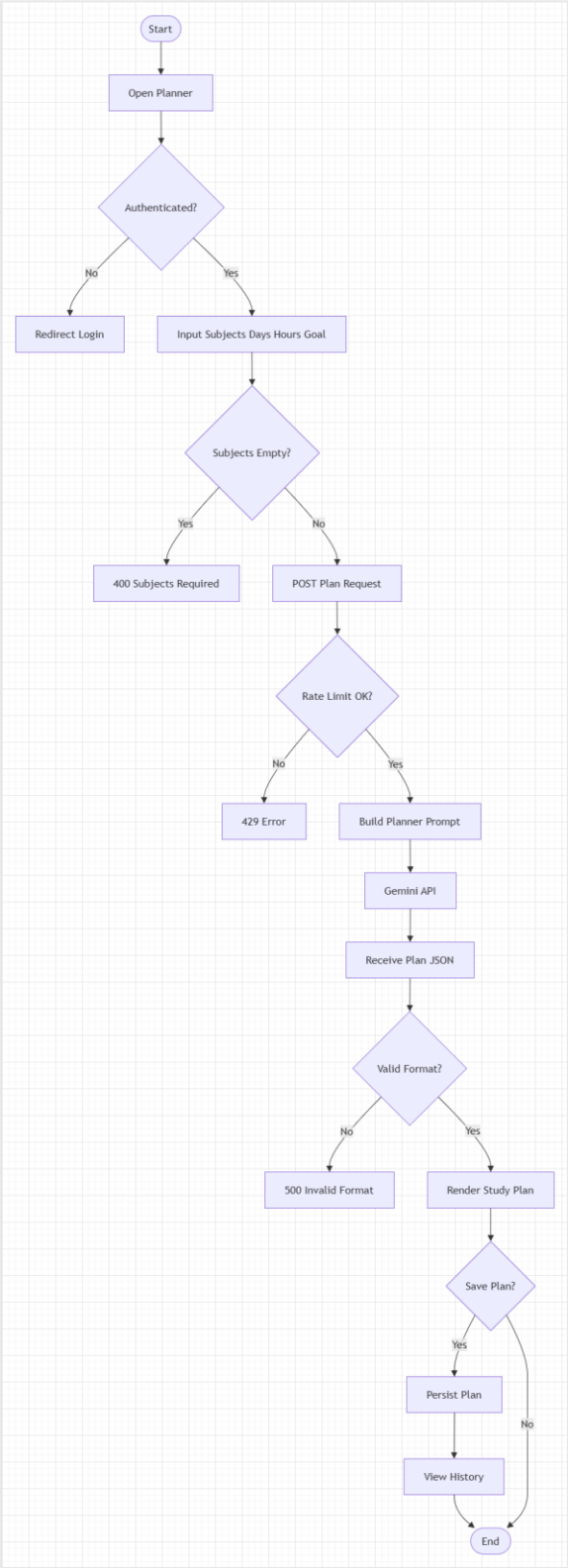


Figure: Activity Diagram — AI Study Planner Workflow

5. API Design Document

All API endpoints are prefixed with /api and return JSON. Authentication is enforced via a Bearer JWT token in the Authorization header for all protected routes. Rate limiting is applied globally (100 req / 15 min) with a stricter AI-specific limit (20 req / 1 min).

5.1 Authentication APIs · Base: /api/auth

Method	Endpoint	Auth	Request Body	Response / Purpose
POST	/api/auth/register	Public	name, email, password, studentId?, department?	201: { token, user } — Register a new student account
POST	/api/auth/login	Public	email, password	200: { token, user } — Authenticate and obtain JWT
GET	/api/auth/me	Protected	—	200: { user } — Return current authenticated user profile

5.2 AI APIs · Base: /api/ai

All AI endpoints require JWT authentication. Each call is proxied to the Google Gemini 2.0 Flash API. An optional apiKey field in the request body allows students to supply their own Gemini API key via the Settings page; if omitted, the server environment key is used.

Method	Endpoint	Request Body	Response / Purpose
POST	/api/ai/chat	messages: [{role, content}], apiKey?	200: { reply } — Multi-turn conversational AI response for academic queries
POST	/api/ai/quiz	topic, numQuestions?, difficulty?, type?, apiKey?	200: { questions[], topic, difficulty, type } — AI-generated quiz in JSON array format
POST	/api/ai/summarize	notes (min 50 chars), subject?, style?, apiKey?	200: { summary, subject, style } — AI-condensed exam-ready note summary
POST	/api/ai/plan	subjects[], days?, hoursPerDay?, goal?, notes?, apiKey?	200: { plan[], subjects, days, hoursPerDay, goal } — Personalised day-by-day study schedule

5.3 Quiz Result APIs · Base: /api/quiz

Method	Endpoint	Auth	Request Body	Response / Purpose
GET	/api/quiz	Protected	—	200: { results[] } — Retrieve all quiz results for authenticated user
POST	/api/quiz	Protected	topic, score, numQ, correct	201: { result } — Save a completed quiz result to history
DELETE	/api/quiz/:id	Protected	—	200: { message } — Remove a specific quiz result by ID

5.4 Study Plan, Summary & Progress APIs

Method	Endpoint	Auth	Request Body	Response / Purpose
GET	/api/plan	Protected	—	200: { plans[] } — List all saved study plans for user
POST	/api/plan	Protected	plan body from AI	201: { plan } — Persist an AI-generated study plan
GET	/api/summary	Protected	—	200: { summaries[] } — List all saved note summaries for user
POST	/api/summary	Protected	summary body from AI	201: { summary } — Save a generated note summary
GET	/api/progress	Protected	—	200: Progress analytics for user (quiz + plan data)
GET	/api/admin/users	Admin Only	—	200: { users[] } — All registered users (minus passwords)
GET	/api/admin/stats	Admin Only	—	200: Platform-wide usage statistics
GET	/api/health	Public	—	200: Server health status, DB connection, version info

6. AI Integration Workflow

6.1 Rationale — Why AI is Needed

University students face three critical, unresolved academic pain points that a traditional static web application cannot address: (1) lack of instant, personalized academic support outside lecture hours; (2) inability to self-assess knowledge through adaptive, on-demand quizzes; (3) time inefficiency from manually consolidating dense lecture notes. Google Gemini 2.0 Flash — a state-of-the-art large language model with a 1-million-token context window and sub-second inference latency — is uniquely positioned to resolve all three at minimal API cost.

6.2 AI Model Selection

Criterion	Detail
Model	Google Gemini 2.0 Flash (gemini-2.0-flash)
Provider	Google AI Studio / Google Generative Language API
Context Window	1,000,000 tokens — supports long lecture notes and multi-turn chat
Inference Speed	Sub-second latency in Flash tier — essential for real-time chatbot UX
Output Quality	Instruction-following, structured JSON output, academic domain accuracy
Cost	Free tier available via Google AI Studio — suitable for academic prototype
Integration	HTTPS REST API — no SDK dependency; compatible with Node.js fetch()
Safety	Built-in safety filters; system instruction controls academic persona

6.3 AI Integration Workflows

6.3.1 AI Academic Chatbot

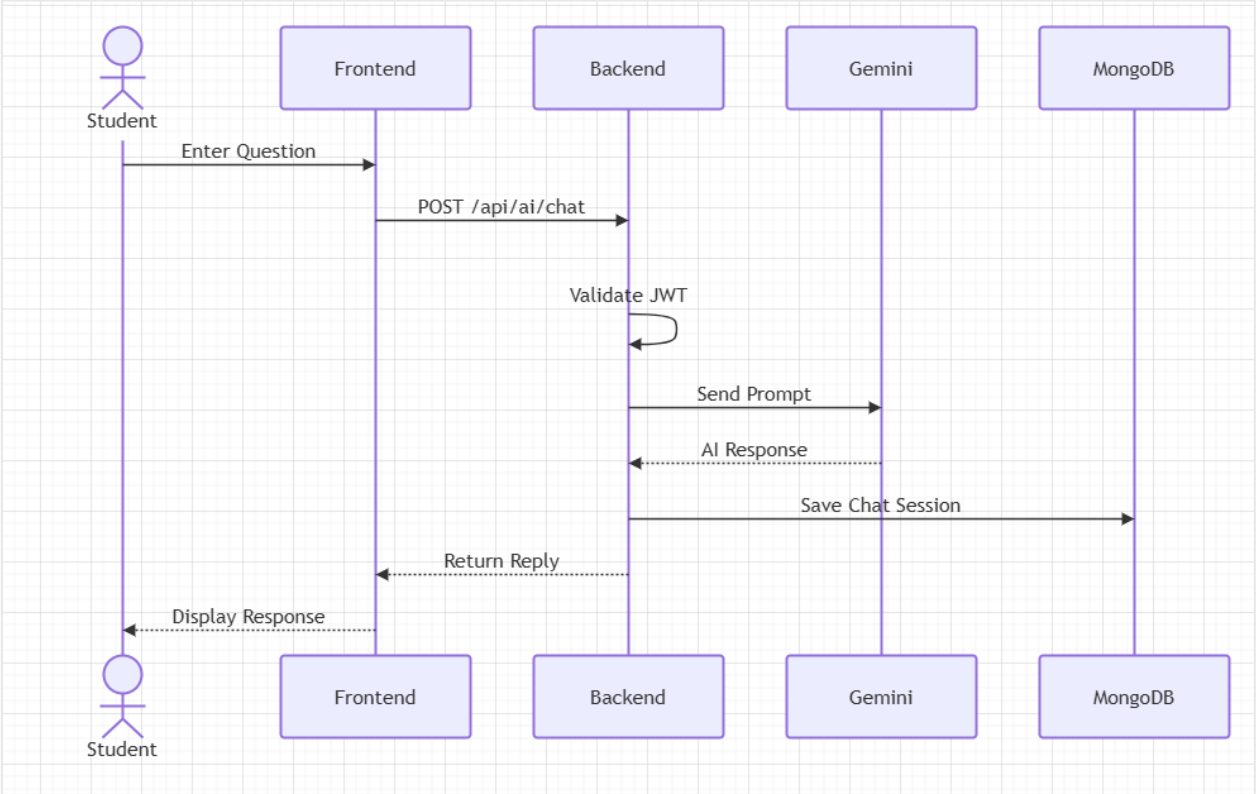


Figure: AI Academic Chatbot

6.3.2 AI Quiz Generator

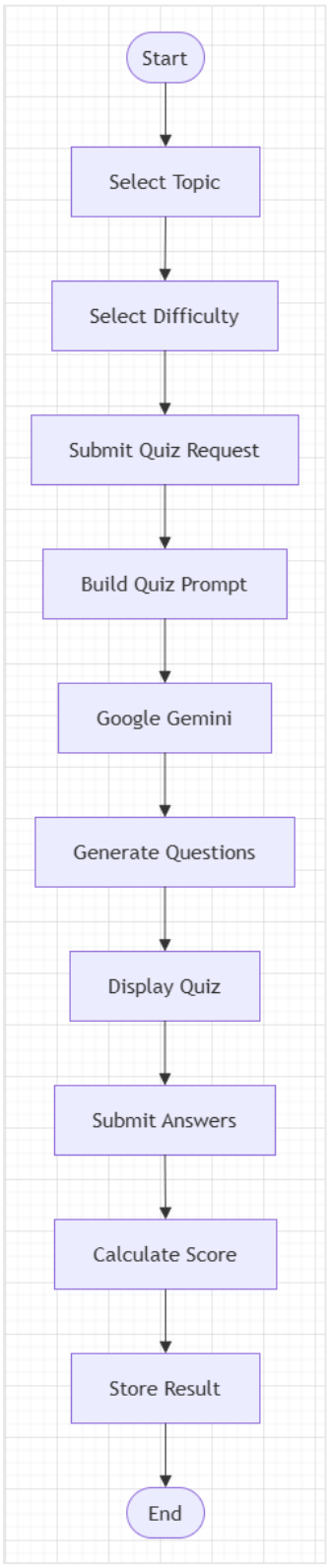


Figure: AI Quiz Generator

6.3.3 AI Note Summarizer

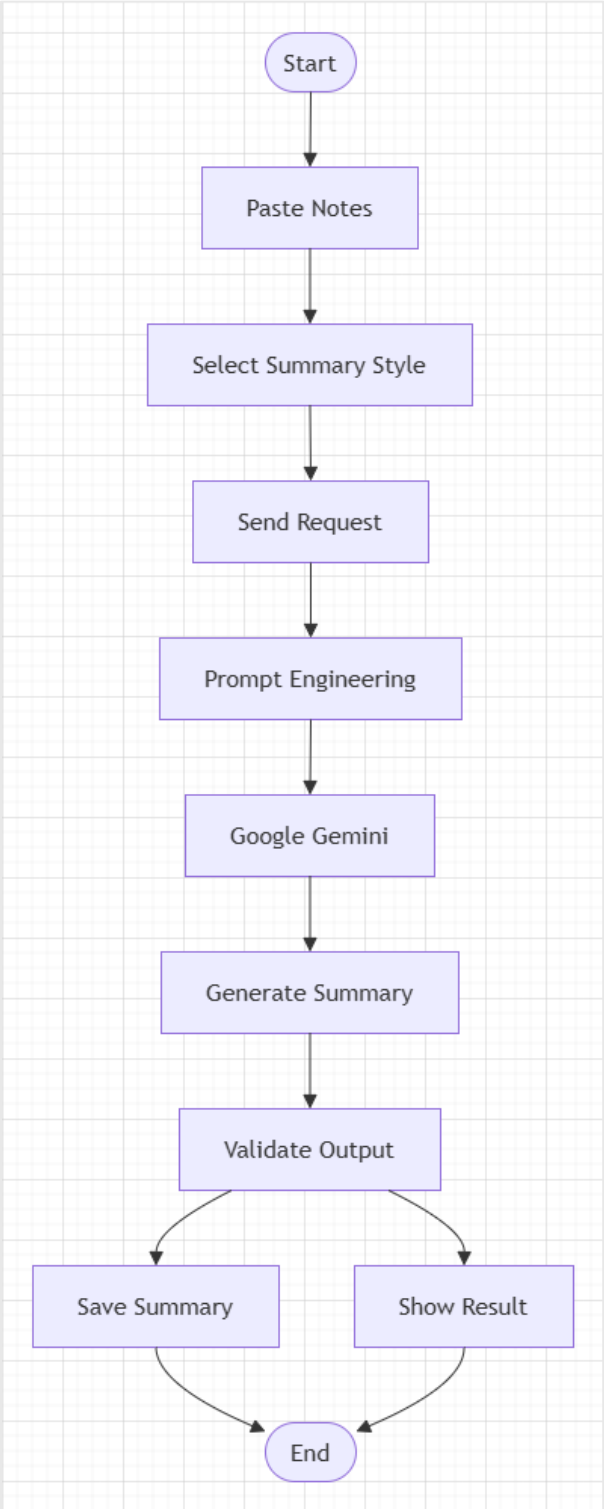


Figure: AI Note Summarizer

6.3.4 AI Study Planner

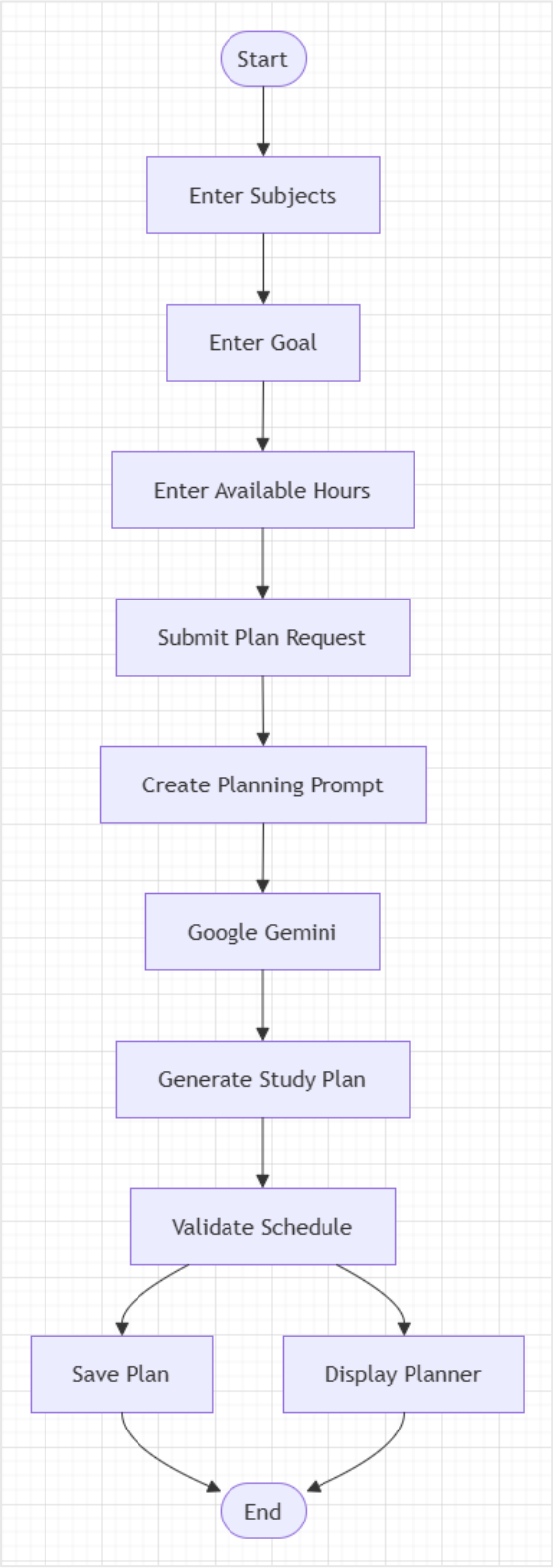


Figure: AI Study Planner

6.4 AI Value Impact Summary

Feature	Without AI	With Gemini AI
Academic Chat	Static FAQ or no support outside office hours	24/7 personalized academic responses with full conversation context
Quiz Generation	Pre-made fixed question banks, stale content	Unlimited on-demand quizzes on any topic at any difficulty level
Note Summarization	Manual highlighting — time-intensive and inconsistent	4-mode AI summarization in seconds; exam-ready output format
Study Planning	Generic timetable templates without personalization	Day-by-day AI plan tailored to student subjects, hours, and goals

7. References

1. Google. (2024). Gemini API Documentation — Generative Language API Reference. Google AI for Developers. https://ai.google.dev/docs
2. Express.js Team. (2024). Express.js v4 — Fast, Unopinionated, Minimalist Web Framework for Node.js. https://expressjs.com
3. Node.js Official Documentation — https://nodejs.org/docs
4. MongoDB, Inc. (2024). MongoDB Manual v7.0 — Document Database Reference Guide. https://www.mongodb.com/docs/manual
5. Mongoose. (2024). Mongoose ODM v8.0 — MongoDB Object Document Mapper for Node.js. https://mongoosejs.com/docs
6. Draw.io / Diagrams.net. (2024). Free Online Diagram Software. https://app.diagrams.net
7. bcrypt npm package — https://www.npmjs.com/package/bcrypt
8. CSE4104-7C-T05 Project Proposal (Week 2) — 25 May 2026
9. CSE4104-7C-T05 Software Requirements Specification (Week 3) — 8 June 2026
10. Week 04 Assignment Instructions — Md. Riaz Mahmud, Assistant Professor, CSE Dept., NUBTK
11. Software_Development_III_Student_Handbook — Md. Riaz Mahmud, Assistant Professor, CSE Dept., NUBTK